

OptArrow Architecture Document

Introduction

OptArrow is an optimization integration engine engineered to bridge optimization clients with solver backends via an efficient, high-performance transport layer. The core runtime capabilities pivot around Python and Julia backends, with supplementary access facilitated through a MATLAB lightweight client interface. The primary objective is to address optimization problems, specifically Linear Programming (LP) and Quadratic Programming (QP). This architectural documentation delineates how requirements are developed, articulated, and transposed into architectural components, following validated project claims.

Stakeholder Engagement and Feedback Collection

Mechanism

OptArrow currently employs GitHub Issues as the primary channel for stakeholder feedback collection. This ensures that users and stakeholders have a direct line to report issues and contribute suggestions to the ongoing development efforts.

Future Prospects

Despite existing mechanisms, formal requirements traceability remains absent, representing critical scope for future enhancement. Implementing a comprehensive traceability system will align stakeholder feedback with decision-making and implementation processes.

Technology Selection and Architectural Decision Rationale

Apache Arrow Selection

OptArrow utilizes Apache Arrow for in-memory data transfer, an essential component for ensuring high performance and enabling cross-language compatibility, particularly between Python and Julia environments. This choice supports expedited data representation transition—converting Python dictionaries to Arrow IPC format, subsequently accessible within Julia for optimized computation.

HiGHS Solver Backend

The HiGHS optimization solver was selected due to its open-source availability and popularity among scientific computing communities. HiGHS provides a robust backend for solving LP and QP problems, integrated directly into the system to deliver scalable computational capabilities.

Evolutionary Architecture and Historical Context

Iterative Architecture Development

OptArrow's architecture has evolved iteratively, favoring flexibility and adaptability over a rigid, predefined roadmap. Such evolution allows seamless integration of stakeholder feedback and internal assessments into subsequent iterations.

Historical Architecture Influence

The COBRA Toolbox informs the historical context of the project, presenting an architectural approach that once influenced OptArrow's development trajectory. Lessons from past implementations inform the current design choice framework.

Design Alternatives and Evaluations

Data Transfer Protocols

No alternative data transfer protocols were trialed against Apache Arrow within the initial selection. Future iterations may benefit from exploring systematic trials to compare performance metrics across other potential protocols.

Design Exploration

Throughout OptArrow's development, diverse design paths have been explored to achieve robustness. Multiple implementation routes were investigated, with iterative adjustments driving progressive enhancements.

Process Maturity and Governance

Architectural Process Gaps

Currently, OptArrow lacks formal architectural governance structures, representing a key area for maturation. Documenting architectural changes and establishing a structured governance process would greatly enhance transparency and scalability.

Requirements Documentation

The maturation of requirements definition and traceability processes is paramount. Establishing formalized documentation and analysis procedures would effectively support decision-making consistency and project reliability.

Future Directions and Recommendations

Enhanced Traceability

We recommend the implementation of an advanced traceability process aligning stakeholder needs with decision-making and implementation documentation, drawing on industry-standard practices such as Architecture Decision Records (ADRs).

Stakeholder Feedback Review

Establishing a formalized process for reviewing stakeholder feedback and deriving actionable insights will support better alignment with user expectations and long-term project

objectives.

Summary of Evidence and Gaps

Evidence Insight

The developer interview complements repository evidence, offering insights into the rationale underscoring technological choices and unveiling existing practices. This evidence illustrates how decisions were made and provides context for missing processes.

Confirmed Gaps

Highlighted gaps include the absence of formal traceability, lack of alternative protocol trials, and nonexistent architectural governance frameworks. Recognition of these gaps serves as a foundation for addressing them in subsequent developments, aligning OptArrow with best practices in the field.

By incorporating these insights, OptArrow's development will be guided by a comprehensive evidence-driven architecture, promoting a sustainable evolution towards meeting stakeholder needs efficiently and effectively.